



Constructing node-independent spanning trees on the line graph of the hypercube by an independent forest scheme

Baolei Cheng^{a,b}, Jianxi Fan^{a,b,*}, Cheng-Kuan Lin^c, Xiaohua Jia^d, Xiaoyan Li^{c,d}

^a School of Computer Science and Technology, Soochow University, Suzhou 215006, China

^b Jiangsu High Technology Research Key Laboratory for Wireless Sensor Networks, Nanjing, Jiangsu 21000, China

^c College of Mathematics and Computer Science, Fuzhou University, Fuzhou 350116, China

^d Department of Computer Science, City University of Hong Kong, Hong Kong

ARTICLE INFO

Article history:

Received 23 August 2018

Received in revised form 17 August 2019

Accepted 20 August 2019

Available online 28 August 2019

Keywords:

Independent forest

Node-independent spanning trees

Node-disjoint paths

Line graph

Network

ABSTRACT

Due to the application in reliable communication, reliable broadcasting, secure message distribution, etc., node/edge-independent spanning trees (ISTs) have attracted much attention in the past twenty years. However, node/edge conjecture is still open for networks with node/edge-connectivity ≥ 5 . So far, results have been obtained on a lot of special networks, but only a few results are reported on the line graphs of them. Hypercubes play important roles in parallel computing systems, and the line graphs of which have been recently adopted for the architectures of data center networks. Since the line graph of n -dimensional hypercube Q_n , $L(Q_n)$, is $(2n-2)$ -regular, whether there exist $2n-2$ node-ISTs rooted at any node on $L(Q_n)$ is an open question. In this paper, we focus on the problem of constructing node-ISTs on $L(Q_n)$. Firstly, we point out that $L(Q_n)$ can be partitioned into 2^{n-1} complete graphs. Then, based on the complete graphs and $n-1$ node-ISTs rooted at 0 on Q_{n-1}^0 , we obtain an “independent forest” containing $2n-2$ trees on $L(Q_n)$. Furthermore, we present an $O(N)$ time algorithm to construct $2n-2$ node-ISTs rooted at node $[0, 2^{n-1}]$ isomorphic to each other on $L(Q_n)$ based on the independent forest, where $N = n \times 2^{n-1}$ is the number of nodes on $L(Q_n)$. In addition, we point out that the $2n-2$ node-ISTs on $L(Q_n)$ is a new method to prove the node/edge-connectivity and the upper bound of $(2n-2)$ -node/edge-wide-diameter of $L(Q_n)$.

© 2019 Elsevier Inc. All rights reserved.

1. Introduction

Node/edge-independent spanning trees (ISTs) can be used in reliable communication protocols [1,24], one-to-all broadcasting [40,47], multi-node broadcasting [5,46], reliable broadcasting [2], and secure message distribution [2]. For example, by using n node/edge-ISTs rooted at the same node u on a communication network G , every tree can use its own encryption method to achieve secure message distribution. Consequently, for any node v other than u in G , the n node/edge-disjoint paths from u to v can transmit their own message securely.

Therefore, it is important to explore how to construct multiple node/edge-ISTs on a network.

In this paper, we focus on the two well-known conjectures on the existence of ISTs on any network [24,50].

Conjecture 1.1. *Given an n -node-connected network G with $n \geq 1$, there exist n node-ISTs rooted at an arbitrary node on G .*

Conjecture 1.2. *Given an n -edge-connected network G with $n \geq 1$, there exist n edge-ISTs rooted at an arbitrary node on G .*

Khuller and Schieber gave a proof that if any n -node-connected network has n node-ISTs, then any n -edge-connected network has n edge-ISTs [25]. However, Gopalan and Ramasubramanian found a counterexample to disprove Khuller and Schieber's result [17]. Thus, either the node conjecture implies the edge conjecture or vice versa is still an open problem. For any network with $n \leq 4$, Conjectures 1.1 and 1.2 were solved [11,12,16,22,24,43,50]. For $n \geq 5$, Conjectures 1.1 and 1.2 have been solved for some restricted classes of networks, such as planar networks [23], product networks [35], hypercubes [38,48], locally twisted cubes [32], crossed cubes [7,9], twisted cubes [4,41], Möbius cubes [8], bijective connection networks [6], even networks [26], odd networks [27], etc.

The line graph has received much attention from researchers in recent years. Many results have been reported on edge-disjoint Hamilton cycles [29], traceability [39], number of spanning trees [14], structural properties [21], topological indices [37], treewidth [20], clique-perfectness [3], etc. Due to the simple structure and suitability for developing algorithms, hypercubes play important

* Corresponding author at: School of Computer Science and Technology, Soochow University, Suzhou 215006, China.
E-mail address: jxfan@suda.edu.cn (J. Fan).

roles in parallel computing systems and a large number of literatures have been obtained on hypercubes and their variants [4,6,8,10,15,30,32,34,36,38,45,48]. Hypercubes belong to generalized hypercubes and can be designed for the architectures of data center networks. Data center networks have attracted the interest from many researchers. Many tasks such as SDN controller assignment [42], network bandwidth allocation [18,19,33], big data transmission [49], information-agnostic mix-flow scheduling [44], etc., have been reported on data center networks. Based on the method in [31], we can obtain a data center network by replacing the nodes in a hypercube with switches and inserting one server into each edge that connects two switches. If we consider the connections among the servers in the data center network, it will be the line graph of the hypercube. Since the n -dimensional hypercube Q_n is n -node-connected and n -edge-connected, the degree of every node in the line graph of Q_n (denoted as $L(Q_n)$) is $2n - 2$. Observing that there are n node-ISTs rooted at any node in Q_n , we naturally ask the following two questions.

1. Do there exist $2n - 2$ node-ISTs rooted at any node on $L(Q_n)$?
2. If there do, how about the performance of the node-ISTs, such as height, structure homoorganicity, etc.?

In this paper, we obtain the following results:

1. We adopt the $n - 1$ node-ISTs rooted at node 0 on Q_{n-1}^0 to construct an independent forest containing $2n - 2$ trees on $L(Q_n)$.
2. Using the above forest, we present an algorithm with time complexity $O(N)$ to construct $2n - 2$ node-ISTs rooted at node $[0, 2^{n-1}]$ on $L(Q_n)$, where $N = n \times 2^{n-1}$ is the number of nodes in $L(Q_n)$.
3. The height of T_i ($i = 1, 2, \dots, 2n - 2$) is $n + 2$ if n is odd, otherwise, $n + 1$, if n is even.
4. For either odd or even n , all $2n - 2$ node-ISTs are isomorphic.

We point out that the $2n - 2$ node-ISTs in $L(Q_n)$ provides a new proof for the node/edge-connectivity and the upper bound of $(2n - 2)$ -node/edge-wide-diameter of $L(Q_n)$.

2. Preliminaries

2.1. Terminology and notations

A network can be abstracted as a graph $G(V(G), E(G))$, where $V(G)$ denotes the node set and $E(G)$ the edge set. The node/edge-connectivity $\kappa(G)/\lambda(G)$ of a connected graph G is the smallest number of nodes/edges whose removal disconnects G . In this paper, graphs and networks are used interchangeably. We will also use decimal numbers to label the nodes in G .

A path started from x and ended at y is denoted as $\langle x, y \rangle$ -path. Two $\langle x, y \rangle$ -paths P and Q are *edge-disjoint* if $E(P) \cap E(Q) = \emptyset$. Two $\langle x, y \rangle$ -paths P and Q are *internally node-disjoint* if they are edge-disjoint and $V(P) \cap V(Q) = \{x, y\}$. Two spanning trees T_1 and T_2 , rooted at the same node u on G , are *edge-independent* if the $\langle u, v \rangle$ -path in T_1 and the $\langle u, v \rangle$ -path in T_2 are edge-disjoint for each $v \in V(G) \setminus \{u\}$. Two spanning trees T_1 and T_2 rooted at u on network G are *node-independent* if the $\langle u, v \rangle$ -path in T_1 and the $\langle u, v \rangle$ -path in T_2 are internally node-disjoint for each $v \in V(G) \setminus \{u\}$. We can also use $\text{path}(u, v, T)$ to denote the $\langle u, v \rangle$ -path in a tree T rooted at node u . A set of spanning trees rooted at the same node on G are *node-independent* (resp., *edge-independent*) if they are pairwise node-independent (resp., edge-independent). We also use node-ISTs (resp., edge-ISTs) for short to represent node-independent spanning trees (resp., edge-independent spanning trees). We extend the definition of edge-disjoint and internally node-disjoint as follows.

Suppose that path P' is started from x' and ended at y , Q' is started from w' and ended at y , and $x' \neq w'$. If $E(P') \cap E(Q') = \emptyset$, P' and Q' are also called *edge-disjoint*. If P' and Q' are edge-disjoint and $V(P') \cap V(Q') = \{y\}$, they are also called *internally node-disjoint*. The *independent forest* is defined as follows.

Suppose that $T_1, T_2, \dots, T_k$ are k trees on a graph G . Let $\text{root}(T)$ denote the root node of a tree T . For any node x in G and any i, j with $1 \leq i < j \leq k$, if the $\langle \text{root}(T_i), x \rangle$ -path in T_i and $\langle \text{root}(T_j), x \rangle$ -path in T_j (if exist) are internally node-disjoint, then we call the set $\{T_1, T_2, \dots, T_k\}$ as an *independent forest*. Furthermore, if $T_1, T_2, \dots, T_k$ are all spanning trees on G , then we call the set $\{T_1, T_2, \dots, T_k\}$ as an *independent spanning forest*.

For the other concepts such as induced subgraph, union, isomorphic, etc., please refer to [6].

2.2. Definition of hypercube and a class of networks—line graphs

We adopt the non-recursive definition of hypercube from [36] as follows.

Definition 2.1. An n -dimensional hypercube Q_n is an undirected graph consisting of 2^n nodes labeled from 0 to $2^n - 1$ and such that there is an edge between any two nodes if and only if the binary representations of their labels differ by one and only one bit.

By Definition 2.1 and the description in [36], we can easily present the recursive definition of hypercube as follows.

Definition 2.2. The 1-dimensional hypercube Q_1 is a complete graph on two nodes whose addresses are 0 and 1. The n -dimensional hypercube Q_n consists of two sub-cubes Q_{n-1}^0 and Q_{n-1}^1 . Q_{n-1}^0 and Q_{n-1}^1 are obtained by prefixing every node in Q_{n-1} with 0 and 1, respectively. The nodes $u = u_{n-1}u_{n-2} \dots u_0 \in Q_{n-1}^0$ and $v = v_{n-1}v_{n-2} \dots v_0 \in Q_{n-1}^1$, where $u_{n-1} = 0$ and $v_{n-1} = 1$, are joined by an edge in Q_n if and only if $u_i = v_i$ for $i = 0, 1, \dots, n - 2$.

Given a node $u = u_{n-1}u_{n-2} \dots u_0$ in Q_n , the neighbor of u in the i -dimension (denoted as $u_{n-1}u_{n-2} \dots u_{i+1}\bar{u}_i u_{i-1}u_{i-2} \dots u_0$) is denoted as $N_i(u)$ for $i=0, 1, \dots, n - 1$, where $N_i(u)$ and u have only one different bit at position i . The parent of a node u in a tree T is denoted as $\text{parent}(u, T)$.

Given a network G , its *line graph* $L(G)$ is a graph such that each node of $L(G)$ represents an edge of G and two nodes of $L(G)$ are adjacent if and only if their corresponding edges share a common endpoint (which are incident) in G . Now we provide Transformation 2.1 to demonstrate the construction of a line graph based on an existing network.

Transformation 2.1. Given a network $G(V(G), E(G))$, we construct the line graph $L(G)$ by the following steps:

- (1) For each edge started from node x and ended at node y in $E(G)$, add a node $[x, y]$ to network $L(G)$, which is referred to as an edge-node.
- (2) For each two adjacent edges (x, y) and (y, z) in G , connect edge-node $[x, y]$ with edge-node $[y, z]$ in $L(G)$.

In $L(G)$, $[x, y]$ and $[y, x]$ denote the same edge-node. In what follows, edge-node and node can be used interchangeably.

In the following, for simplicity, we can also use decimal numbers to represent binary strings, i.e., we use decimal integer 15 to represent binary string 1111.

Obviously, $L(Q_n)$ is a $(2n - 2)$ -regular graph and it has $n \times 2^{n-1}$ nodes and $n \times (n - 1) \times 2^{n-1}$ edges.

2.3. Stars in n -dimensional hypercube

To facilitate our discussion, we classify the nodes of Q_n into two types, and draw them in black and white respectively. The black nodes are those whose labels contain even number of 1's, while the white nodes are those whose labels contain odd

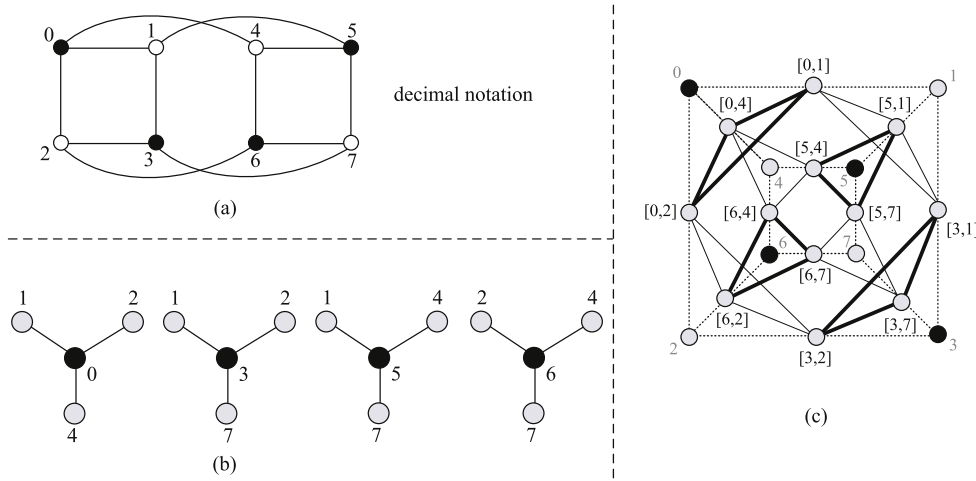


Fig. 1. (a) 3-Dimensional hypercube Q_3 ; (b) Four stars in Q_3 ; (c) Four complete graphs in $L(Q_3)$.

number of 1's. We denote A the node set of even 1's, and B the node set of odd 1's. For example, in Q_3 , we have $A = \{0, 3, 5, 6\}$ and $B = \{1, 2, 4, 7\}$. Fig. 1(a) shows the examples of Q_3 with black/white nodes. It is well known that Q_n is a bipartite graph, and it can be seen that the black nodes just fall in one part of the bipartite, while the white nodes fall in the other.

Any node in the black set and its neighbors in the white set forms a star and vice versa. Then, we have the following observation.

Observation 2.1. For Q_n and its line graph $L(Q_n)$, the following results hold:

- (1) There are 2^{n-1} stars in Q_n with the internal black nodes and the internal white nodes, respectively.
- (2) Given any node u in Q_n , $[u, N_0(u)]$, $[u, N_1(u)]$, ..., $[u, N_{n-1}(u)]$ in $L(Q_n)$ form a complete graph.
- (3) $L(Q_n)$ can be partitioned into 2^{n-1} complete graphs.

Every star in G means a complete graph $L(G)$. In Q_3 , there are 4 stars with the internal black nodes 0, 3, 5, 6, respectively. See Fig. 1(b). The corresponding 4 complete graphs are with the node sets $\{[0,1], [0,2], [0,4]\}$, $\{[3,1], [3,2], [3,7]\}$, $\{[5,1], [5,4], [5,7]\}$, $\{[6,2], [6,4], [6,7]\}$ and edge sets $\{([0,1],[0,2]), ([0,1],[0,4]), ([0,2],[0,4])\}$, $\{([3,1],[3,2]), ([3,1],[3,7]), ([3,2],[3,7])\}$, $\{([5,1],[5,4]), ([5,1],[5,7]), ([5,4],[5,7])\}$, $\{([6,2],[6,4]), ([6,2],[6,7]), ([6,4],[6,7])\}$, respectively. See the 4 complete graphs with bold lines in Fig. 1(c).

3. Node-independent spanning trees rooted at any node on $L(Q_n)$

In this section, we first analyze the construction of node-ISTs on Q_n from [10], which can be obtained by n iterations. Secondly, based on the node-ISTs on Q_n , we propose a method to construct the independent forest containing $2n - 2$ trees on $L(Q_n)$. Then, by using the above forest as proper graphs, we give an algorithm to construct node-ISTs on $L(Q_n)$.

3.1. Analysis of construction method of node-ISTs on Q_n

We first analyze the construction of the node-ISTs on Q_n , which will be adopted from [10]. In [10], the node-ISTs on Q_n can be constructed based on circular dimensional-permutation (CDP). By this method, every node-ISTs can be constructed by n

iterations. Fig. 2(a) shows the CDPs

- 0, 1, 2
- 0, 1, 2, 3
- ...
- 0, 1, ..., $n - 2$, $n - 1$,

respectively.

The construction procedures of every node-IST rooted at node u comes down to n iterations. In the initialization, there are nodes $N_0(u)$, $N_1(u)$, ..., $N_{n-1}(u)$ in trees T_1 , T_2 , ..., T_n , respectively. T_1 , T_2 , ..., T_n take the dimensional-permutation

- 1, 2, ..., $n - 1$, 0
- 2, 3, ..., $n - 1$, 0, 1
- ...
- $n - 1$, 0, 1, ..., $n - 2$
- 0, 1, 2, ..., $n - 1$

respectively. During every iteration, each node in every tree connect to its neighbor of the same dimension in the corresponding dimensional-permutation.

Fig. 2(b) shows the construction of 4 node-ISTs rooted at 0 on Q_4 . Every node-IST is constructed by 4 iterations based on the CDP 0, 1, 2, 3.

Since Q_n is node-symmetric and edge-symmetric, it is easy to verify that its line graph, $L(Q_n)$, is also node-symmetric. We only need to construct the $2n - 2$ node-ISTs rooted at node $[0, 2^{n-1}]$ on $L(Q_n)$. The $n - 1$ node-ISTs $T_{A,1}$, $T_{A,2}$, ..., $T_{A,n-1}$ rooted at 0 on Q_{n-1}^0 can be obtained in $O(N)$ time by the algorithm in parallel mode in [10], where $N = 2^n$. By adding 2^{n-1} to every node in $T_{A,1}$, $T_{A,2}$, ..., $T_{A,n-1}$, we obtain another set of $n - 1$ node-ISTs $T_{B,1}$, $T_{B,2}$, ..., $T_{B,n-1}$ rooted at 2^{n-1} on Q_{n-1}^1 . [48] gave another construction method of node-ISTs in Q_n .

3.2. Construction algorithm of node-ISTs in the line graph $L(Q_n)$

Based on the $n - 1$ node-ISTs on Q_{n-1} obtained by [10], we can easily construct the $n - 1$ node-ISTs on Q_{n-1}^0 . Further, we present an algorithm to construct an independent forest containing $2n - 2$ trees on $L(Q_n)$, based on the given $n - 1$ node-ISTs $T_{A,1}$, $T_{A,2}$, ..., $T_{A,n-1}$ rooted at 0 on Q_{n-1}^0 . The forest will be used to construct node-ISTs on $L(Q_n)$ in Section 3.2.3.

3.2.1. Construction algorithm of an independent forest on $L(Q_n)$

Here, we first present an algorithm, called IndependentForest, in which we construct an independent forest containing $2n - 2$ trees on $L(Q_n)$, based on the given $n - 1$ node-ISTs $T_{A,1}$, $T_{A,2}$, ..., $T_{A,n-1}$ rooted at 0 on Q_{n-1}^0 .

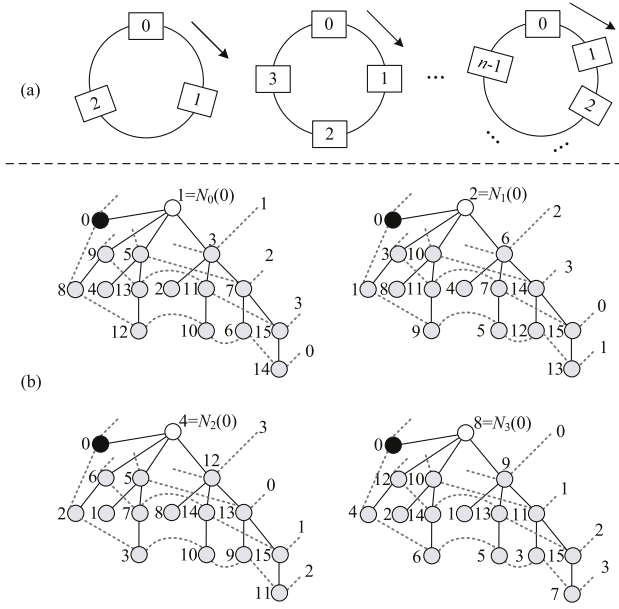


Fig. 2. (a) Circular dimensional-permutations (CDPs); (b) Four node-ISTs rooted at 0 on Q_4 based on the CDP 0, 1, 2, 3.

Example 1. Based on the CDP 0, 1, we can obtain two node-ISTs $T_{A,1}$ and $T_{A,2}$ rooted at 0 on Q_2^0 with $V(T_{A,1}) = V(T_{A,2}) = \{0, 1, 2, 3\}$, $E(T_{A,1}) = \{(0,1), (1,3), (3,2)\}$, and $E(T_{A,2}) = \{(0,2), (2,3), (3,1)\}$. The independent forest is constructed as follows.

Step 1. By adding $2^2 = 4$ to every node in $T_{A,1}$ and $T_{A,2}$, we obtain two node-ISTs $T_{B,1}$ and $T_{B,2}$ rooted at 0 in Q_2^1 with $V(T_{B,1}) = V(T_{B,2}) = \{4, 5, 6, 7\}$, $E(T_{B,1}) = \{(4,5), (5,7), (7,6)\}$, and $E(T_{A,2}) = \{(4,6), (6,7), (7,5)\}$.

Let $T_{A,3} = T_{A,1}$, $T_{A,4} = T_{A,2}$, $T_{B,3} = T_{B,1}$, and $T_{B,4} = T_{B,2}$. See Fig. 3(a).

Step 2. Construct tree T_1'' , T_2'' , T_3'' , and T_4'' as follows.

Disconnect all the edges between internal nodes in $T_{B,i}$ and connect all internal nodes in $T_{A,i}$ with $T_{B,i}$ to construct T_i'' for $i = 1, 2$. Disconnect all the edges between internal nodes in $T_{A,i}$ and connect all internal nodes in $T_{B,i}$ with $T_{A,i}$ to construct T_i'' for $i = 3, 4$, noticing that the direction is contrary to T_1 and T_2 . See Fig. 3(b).

Step 3. Construct tree T_1' , T_2' , T_3' , and T_4' as follows.

Use Transformation 2.1 on T_1'' , T_2'' , T_3'' , and T_4'' . For any internal edge (x, y) (suppose that x is closer to the root node) in $T_{B,i}$, connect node $[x, y]$ with nodes $[x, x - 2^{n-1}]$, for $i = 1, 2$ and $n = 3$. For any internal edge (x, y) (suppose that x is closer to the root node) in $T_{A,i}$, connect node $[x, y]$ with node $[x, x + 2^{n-1}]$, for $i = 3, 4$ and $n = 3$.

See Fig. 3(c). Clearly, T_1' , T_2' , T_3' , and T_4' form an independent forest on $L(Q_3)$.

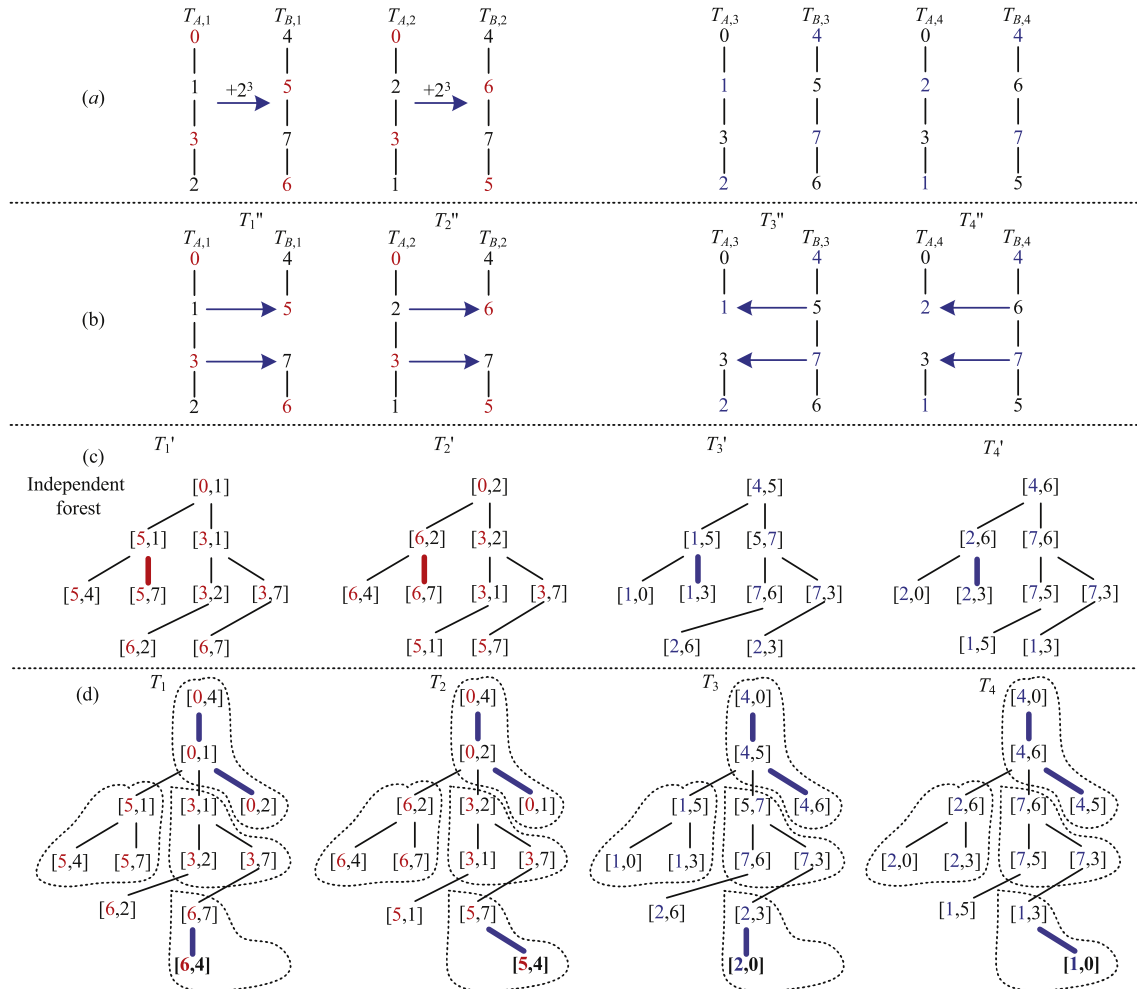


Fig. 3. (a) Step 1; (b) Step 2; (c) Step 3 of Algorithm IndependentForest; (d) 4 node-ISTs rooted at $[0,4]$ on $L(Q_3)$.

Algorithm 1 IndependentForest.**Input:** $n - 1$ node-ISTs $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$ rooted at 0 on Q_{n-1}^0 .**Output:** A forest containing $2n - 2$ trees on $L(Q_n)$.**Begin****Step 1:**

- 1: **for** $k = 1$ to $n - 1$ **do**
- 2: **in parallel**
- 3: Construct tree $T_{B,k}$ rooted at 2^{n-1} on Q_{n-1}^1 , such that $T_{B,k}$ is constructed of $T_{A,k}$ by updating every node x in $T_{A,k}$ as $x + 2^{n-1}$.
- 4: Let $T_{B,n+k} = T_{B,k}$. //Duplicate $T_{B,k}$ as $T_{B,n+k}$.
- 5: Let $T_{A,n+k} = T_{A,k}$. //Duplicate $T_{A,k}$ as $T_{A,n+k}$.

end for**Step 2:**

- 7: **for** $k = 1$ to $n - 1$ **do**
- 8: **in parallel**
- 9: Construct tree T_k'' as follows.
- 10: Disconnect all the edges between internal nodes in $T_{B,k}$.
- 11: Connect all the internal nodes in $T_{A,k}$ with its neighbor nodes in $T_{B,k}$.
- 12: Construct trees T_{k+n-1}'' as follows.
- 13: Disconnect all the edges between internal nodes in $T_{A,k+n-1}$.
- 14: Connect all the internal nodes in $T_{B,k+n-1}$ with its neighbor nodes in $T_{A,k+n-1}$.

end for**Step 3:**

- 15: **for** $k = 1$ to $n - 1$ **do**
- 16: **in parallel**
- 17: Construct tree T_k' as follows.
- 18: **Step 3.1:** Use Transformation 2.1 on T_k'' and T_{k+n-1}'' .
- 19: **Step 3.2:** For any edge (x, y) between the internal nodes in $T_{B,k}$ (Suppose that x is closer to the root node 2^{n-1}), connect node $[x, y]$ with node $[x, x - 2^{n-1}]$ in T_k'' .

For any edge (x, y) between the internal nodes in $T_{A,k}$ (Suppose that x is closer to the root node 0), connect node $[x, y]$ with node $[x, x + 2^{n-1}]$ in T_{k+n-1}'' .

- 20: We obtain T_k' .
- 21: We obtain T_{k+n-1}' .

end for**End****3.2.2. Correctness of algorithm independent forest on $L(Q_n)$**

Based on Algorithm IndependentForest, the following two lemmas are clear.

Lemma 3.1. $T_{B,1}, T_{B,2}, \dots, T_{B,n-1}$ are $n - 1$ node-ISTs rooted at 2^{n-1} on Q_{n-1}^1 and $T_{B,1}, T_{B,2}, \dots, T_{B,n-1}$ are isomorphic to $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$, respectively.

Lemma 3.2. T_i'' is a spanning tree on Q_n for $i = 1, 2, \dots, 2n - 2$.

In what follows, we will prove that $T_1'', T_2'', \dots, T_{2n-2}''$ obtained by Algorithm IndependentForest satisfy the following property and the property will be used to prove that $T_1', T_2', \dots, T_{2n-2}'$ obtained by Algorithm IndependentForest form an independent forest.

Lemma 3.3. For any node $v \in V(Q_n) \setminus \{0\}$ and any i, j with $1 \leq i < j \leq 2n - 2$, we have

- (1) $V(\text{path}(v, \text{root}(T_i''), T_i'')) \cap V(\text{path}(v, \text{root}(T_j''), T_j'')) \subseteq \{\text{root}(T_i'') = \text{root}(T_j''), v, \text{parent}(v, T_i'')\}$ if $\text{parent}(v, T_i'') = \text{parent}(v, T_j'')$. Otherwise, $V(\text{path}(v, \text{root}(T_i''), T_i'')) \cap V(\text{path}(v, \text{root}(T_j''), T_j'')) \subseteq \{\text{root}(T_i''), \text{root}(T_j''), v\}$.

- (2) $V(\text{path}(v, \text{root}(T_i''), T_i'')) \cup V(\text{path}(v, \text{root}(T_j''), T_j'')) \supseteq \{\text{parent}(v, T_i''), \text{root}(T_j''), v, \text{parent}(v, T_i'')\}$ if $\text{parent}(v, T_i'') = \text{parent}(v, T_j'')$. Otherwise, $V(\text{path}(v, \text{root}(T_i''), T_i'')) \cup V(\text{path}(v, \text{root}(T_j''), T_j'')) \supseteq \{\text{root}(T_i''), \text{root}(T_j''), v, \text{parent}(v, T_i''), \text{parent}(v, T_j'')\}$.

Proof. By Algorithm IndependentForest, $v \in V(T_{A,i}) = V(T_{A,j})$ or $v \in V(T_{B,i}) = V(T_{B,j})$, where $1 \leq i < j \leq 2n - 2$. We have the following cases.

Case 1. $1 \leq i < j \leq n - 1$. Based on Algorithm IndependentForest, $\text{root}(T_i'') = \text{root}(T_j'') = 0$. We have the following subcases.

Case 1.1. $v \in V(T_{A,i})$. Since $T_{A,i}$ and $T_{A,j}$ are independent, then we can easily verify that the lemma holds.

Case 1.2. $v \in V(T_{B,i})$. Then, we have the following subcases.

Case 1.2.1. $\text{parent}(v, T_i'') \in T_{A,i}$. We have the following subcases.

Case 1.2.1.1. $\text{parent}(v, T_j'') \in T_{A,j}$. By Algorithm IndependentForest, $\text{parent}(v, T_i'') = \text{parent}(v, T_j'')$. Since $T_{A,i}$ and $T_{A,j}$ are node-independent, we have

- (1) $V(\text{path}(v, 0, T_i'')) \cap V(\text{path}(v, 0, T_j'')) \subseteq \{0, v, \text{parent}(v, T_i'')\}$, and

- (2) $V(\text{path}(v, 0, T_i'')) \cup V(\text{path}(v, 0, T_j'')) \supseteq \{0, v, \text{parent}(v, T_i'')\}$.

Case 1.2.1.2. $\text{parent}(v, T_j'') \in T_{B,j}$. By Algorithm IndependentForest, $V(\text{path}(v, 0, T_i'')) = V(\text{path}(v - 2^{n-1}, 0, T_{A,i})) \cup \{v\}$ and $V(\text{path}(v, 0, T_j'')) = V(\text{path}(v - 2^{n-1}, 0, T_{A,j})) \cup \{v, \text{parent}(v, T_{B,j})\} \setminus \{v - 2^{n-1}\}$. Since $T_{A,i}$ and $T_{A,j}$ are node-independent, we have

- (1) $V(\text{path}(v, 0, T_i'')) \cap V(\text{path}(v, 0, T_j'')) \subseteq \{0, v\}$, and
- (2) $V(\text{path}(v, 0, T_i'')) \cup V(\text{path}(v, 0, T_j'')) \supseteq \{0, v, \text{parent}(v, T_i''), \text{parent}(v, T_j'')\}$.

Case 1.2.2. $\text{parent}(v, T_i'') \in T_{B,i}$. We have the following subcases.

Case 1.2.2.1. $v = 2^{n-1}$. Then, $\text{path}(v, 0, T_i'')$ contains nodes $0, 0+2^{i-1}, 0+2^{i-1}+2^{n-1}, v = 2^{n-1}$ and $\text{path}(v, 0, T_j'')$ contains nodes $0, 0+2^{j-1}, 0+2^{j-1}+2^{n-1}, v = 2^{n-1}$. We can verify that the lemma holds.

Case 1.2.2.2. $v \neq 2^{n-1}$ and $\text{parent}(v, T_j'') \in T_{A,j}$. By Algorithm IndependentForest, $V(\text{path}(v, 0, T_j'')) = V(\text{path}(v - 2^{n-1}, 0, T_{A,j})) \cup \{v\}$ and $V(\text{path}(v, 0, T_i'')) = V(\text{path}(v - 2^{n-1}, 0, T_{A,i})) \cup \{v, \text{parent}(v, T_{B,i})\} \setminus \{v - 2^{n-1}\}$. Since $T_{A,i}$ and $T_{A,j}$ are node-independent, we have

- (1) $V(\text{path}(v, 0, T_i'')) \cap V(\text{path}(v, 0, T_j'')) \subseteq \{0, v\}$, and
- (2) $V(\text{path}(v, 0, T_i'')) \cup V(\text{path}(v, 0, T_j'')) \supseteq \{0, v, \text{parent}(v, T_i''), \text{parent}(v, T_j'')\}$.

Case 1.2.2.3. $v \neq 2^{n-1}$ and $\text{parent}(v, T_j'') \in T_{B,j}$. By Algorithm IndependentForest, it is easy to verify v is a leaf node in both $T_{B,i}$ and $T_{B,j}$. By Algorithm IndependentForest, $V(\text{path}(v, 0, T_i'')) = V(\text{path}(v - 2^{n-1}, 0, T_{A,i})) \cup \{v, \text{parent}(v, T_{B,i})\} \setminus \{v - 2^{n-1}\}$ and $V(\text{path}(v, 0, T_j'')) = V(\text{path}(v - 2^{n-1}, 0, T_{A,j})) \cup \{v, \text{parent}(v, T_{B,j})\} \setminus \{v - 2^{n-1}\}$. Since $T_{A,i}$ and $T_{A,j}$ are node-independent, we have

- (1) $V(\text{path}(v, 0, T_i'')) \cap V(\text{path}(v, 0, T_j'')) \subseteq \{0, v\}$, and
- (2) $V(\text{path}(v, 0, T_i'')) \cup V(\text{path}(v, 0, T_j'')) \supseteq \{0, v, \text{parent}(v, T_i''), \text{parent}(v, T_j'')\}$.

Case 2. $n \leq i < j \leq 2n - 2$. By Lemma 3.1, $T_{B,1}, T_{B,2}, \dots, T_{B,n-1}$ are $n - 1$ node-ISTs rooted at 2^{n-1} on Q_{n-1}^1 and $T_{B,1}, T_{B,2}, \dots, T_{B,n-1}$ are isomorphic to $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$, respectively. Further, since the Step 2 in Algorithm IndependentForest is similar to the Step 1 in Algorithm IndependentForest, thus the proof is similar to Case 1.

Case 3. $0 \leq i \leq n - 1$ and $n \leq j \leq 2n - 2$. We have the following subcases.

Case 3.1. $v \in V(T_{A,i})$. By Algorithm IndependentForest, $V(\text{path}(v, 0, T_i'')) = V(\text{path}(v, 0, T_{A,i}))$. We have the following subcases.

Case 3.1.1. v is a leaf node in $T_{A,j}$. We have the following subcases.

Case 3.1.1.1. $\text{parent}(v, T_{A,i}) = \text{parent}(v, T_{A,j})$. Then, $V(\text{path}(v, 2^{n-1}, T_j'')) \cap V(\text{path}(v, 0, T_{A,i})) \subseteq \{0, v, \text{parent}(v, T_{A,i})\}$ and $V(\text{path}(v, 0, T_{A,i})) \cup V(\text{path}(v, 0, T_{A,j})) \supset \{0, 2^{n-1}, v, \text{parent}(v, T_{A,i})\}$.

Case 3.1.1.2. $\text{parent}(v, T_{A,i}) \neq \text{parent}(v, T_{A,j})$ and $\text{parent}(v, T_{A,j}) \in V(T_{B,j})$. By Algorithm IndependentForest, $V(\text{path}(v, 2^{n-1}, T_j'')) \cap V(\text{path}(v, 0, T_{A,i})) \subseteq \{0, 2^{n-1}, v\}$ and $V(\text{path}(v, 0, T_{A,i})) \cup V(\text{path}(v, 0, T_{A,j})) \supset \{0, 2^{n-1}, v, \text{parent}(v, T_{A,i}), \text{parent}(v, T_{A,j})\}$.

Case 3.1.2. v is not a leaf node in $T_{A,j}$. Then, $V(\text{path}(v, 2^{n-1}, T_j'')) \cap V(\text{path}(v, 0, T_{A,i})) = \{v\}$ and $V(\text{path}(v, 2^{n-1}, T_j'')) \cup V(\text{path}(v, 0, T_{A,i})) \subseteq \{0, 2^{n-1}, v, \text{parent}(v, T_{A,i}), \text{parent}(v, T_{A,j})\}$.

Case 3.2. $v \in V(T_{B,i})$. The proof is similar to **Case 3.1**.

In summary, the lemma holds. $\square$

Based on **Transformation 2.1** and **Step 3** of Algorithm IndependentForest, $T_1', T_2', \dots, T_{2n-2}'$ can be transformed into $T_1', T_2', \dots, T_{2n-2}'$. By **Lemma 3.3**, we have the following lemma.

Lemma 3.4. For any v in Q_n , any $1 \leq i < j \leq 2n - 2$, $V(\text{path}(\text{root}(T_i'), [v, \text{parent}(v, T_i'')], T_i')) \cap V(\text{path}(\text{root}(T_j'), [v, \text{parent}(v, T_j'')], T_j')) = \emptyset$ and $V(\text{path}(\text{root}(T_i'), [v, \text{parent}(v, T_i'')], T_i')) \cup V(\text{path}(\text{root}(T_j'), [v, \text{parent}(v, T_j'')], T_j')) \supseteq \{\text{root}(T_i'), \text{root}(T_j')\}$.

Proof. Suppose that the trees obtained by **Step 3.1** in Algorithm IndependentForest are $T_1'^{(*)}, T_2'^{(*)}, \dots, T_{2n-2}'^{(*)}$. The root nodes of $T_1'^{(*)}, T_2'^{(*)}, \dots, T_{2n-2}'^{(*)}$ are $[0, 1], [0, 2^1], \dots, [0, 2^{n-2}], [2^{n-1}, 2^{n-1}+1], [2^{n-1}, 2^{n-1}+2^1], \dots, [2^{n-1}, 2^{n-1}+2^{n-2}]$, respectively. By **Lemma 3.3**, we can verify that the following claim holds.

Claim 1. For any v in Q_n , any $1 \leq i < j \leq 2n - 2$, $V(\text{path}(\text{root}(T_i'^{(*)}), [v, \text{parent}(v, T_i'')], T_i'^{(*)})) \cap V(\text{path}(\text{root}(T_j'^{(*)}), [v, \text{parent}(v, T_j'')], T_j'^{(*)})) = \emptyset$ and $V(\text{path}(\text{root}(T_i'^{(*)}), [v, \text{parent}(v, T_i'')], T_i'^{(*)})) \cup V(\text{path}(\text{root}(T_j'^{(*)}), [v, \text{parent}(v, T_j'')], T_j'^{(*)})) \supseteq \{[0, 2^{i-1}], [0, 2^{j-1}]\}$.

Now we only need to consider the new nodes created by **Step 3.2** in Algorithm IndependentForest. We have the following cases.

Case 1. $1 \leq i < j \leq n - 1$. Without loss of generality, suppose that $[v, \text{parent}(v, T_i'')]$ is a new node, which implies that $(v, \text{parent}(v, T_i''))$ is an internal edge in $T_{B,i}$ and $(v, \text{parent}(v, T_j''))$ is an edge in $T_{B,j}$.

Since $T_{B,i}$ and $T_{B,j}$ are node-independent, suppose that the path from 2^{n-1} to v in $T_{B,i}$ contains the edges $(2^{n-1}, a_1), (a_1, a_2), (a_2, a_3), \dots, (a_{k-1}, a_k = v)$ and the path from 2^{n-1} to y in $T_{B,j}$ contains the edges $(2^{n-1}, b_1), (b_1, b_2), (b_2, b_3), \dots, (b_{l-1}, b_l = v)$. The above two paths are node-disjoint. Then, we suppose that the path from $[0, 2^{i-1}]$ to $[v, \text{parent}(v, T_i'')]$ in T_i' is

$P: [0, 2^{i-1}], [2^{i-1}, a_1 - 2^{n-1}], [a_1 - 2^{n-1}, a_2 - 2^{n-1}], [a_2 - 2^{n-1}, a_3 - 2^{n-1}], \dots, [a_{k-1} - 2^{n-1}, a_k - 1], [a_k - 1, v]$.

The path from $[0, 2^{j-1}]$ to $[v, \text{parent}(v, T_j'')]$ in T_j' is

$Q: [0, 2^{j-1}], [2^{j-1}, b_1 - 2^{n-1}], [b_1 - 2^{n-1}, b_2 - 2^{n-1}], [b_2 - 2^{n-1}, b_3 - 2^{n-1}], \dots, [b_{l-1} - 2^{n-1}, b_l - 1], [b_l - 1, v]$. Then $P \cap Q = \emptyset$ and $P \cup Q \supset \{[0, 2^{i-1}], [0, 2^{j-1}]\}$.

Case 2. $n \leq i < j \leq 2n - 2$. Since the construction of $T_n'', T_{n+1}'', \dots, T_{2n-2}''$ is similar to $T_1'', T_2'', \dots, T_{n-1}''$ (Lines 16–19 in Algorithm IndependentForest), respectively, the proof is similar to that in **Case 1**.

Case 3. $1 \leq i \leq n - 1$ and $n \leq j \leq 2n - 2$. We have the following subcases.

Case 3.1. $(v, \text{parent}(v, T_i''))$ is an internal edge in $T_{B,i}$.

Suppose that the path from 2^{n-1} to v in $T_{B,i}$ contains the edges $(2^{n-1}, a_1), (a_1, a_2), (a_2, a_3), \dots, (a_{k-1}, a_k = v)$ and the path from 2^{n-1} to v in $T_{B,j}$ contains the edges $(2^{n-1}, b_1 = 2^{n-1} + 2^{j-n}), (b_1, b_2), (b_2, b_3), \dots, (b_{l-1}, b_l = v)$. Then, we suppose that the path from $[0, 2^{i-1}]$ to $[v, \text{parent}(v, T_i'')]$ in T_i' is

$P: [0, 2^{i-1}], [2^{i-1}, a_1 - 2^{n-1}], [a_1 - 2^{n-1}, a_2 - 2^{n-1}], [a_2 - 2^{n-1}, a_3 - 2^{n-1}], \dots, [a_{k-1} - 2^{n-1}, a_k - 1], [a_k - 1, v]$.

The path from $[2^{n-1}, b_1 = 2^{n-1} + 2^{j-1}]$ to $(v, \text{parent}(v, T_j''))$ in T_j' is

$Q: [2^{n-1}, b_1], [b_1, b_2], [b_2, b_3], \dots, [b_{l-1} - 2^{n-1}, b_l - 1], [b_l - 1, v]$.

Here, $b_1, b_2, \dots, b_{l-1} > 2^{n-1}$, while $a_1 - 2^{n-1}, a_2 - 2^{n-1}, \dots, a_{k-1} - 2^{n-1} < 2^{n-1}$, which implies that $P \cap Q = \emptyset$ and $P \cup Q \supset \{[0, 2^{i-1}], [2^{n-1}, 2^{n-1} + 2^{j-1}]\}$.

Case 3.2. $(v, \text{parent}(v, T_i''))$ is an internal edge in $T_{A,j}$. The proof is similar to that in **Case 3.1**.

Hence, the lemma holds. $\square$

Based on **Lemma 3.4**, we have the following theorem.

Theorem 3.1. $T_1', T_2', \dots, T_{2n-2}'$ form an independent forest on $L(Q_n)$ containing $2n - 2$ trees.

3.2.3. Construction algorithm of independent spanning trees on $L(Q_n)$

Given a node u in Q_n , the complete graph containing the common member u in $L(Q_n)$ is the graph with the node set $\{[u, N_0(u)], [u, N_1(u)], \dots, [u, N_{n-1}(u)]\}$, and each pair of nodes in the graph is connected by an edge. Based on the construction rule discussed in the above section, we now present an algorithm, called NodeIST, to construct $2n - 2$ node-ISTs $T_1, T_2, \dots, T_{2n-2}$ rooted at node $[0, 2^{n-1}]$ on $L(Q_n)$, based on the $n - 1$ node-ISTs $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$ rooted at 0 on Q_{n-1}^0 . Here, we call Algorithm IndependentForest to construct the subtrees in every node-IST.

By Algorithm NodeIST, given node-ISTs $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$ rooted at 0 on Q_{n-1}^0 , firstly, we call Algorithm IndependentForest to construct an independent forest on $L(Q_n)$. Secondly, we generate the $2n - 2$ node-ISTs rooted at $[0, 2^{n-1}]$. Based on the independent forest in **Fig. 3(c)**, the 4 node-ISTs rooted at node $[0, 4]$ constructed by Algorithm NodeIST are shown in **Fig. 3(d)**.

Now we provide the following example to show the construction of node-ISTs on $L(Q_4)$.

Example 2. **Fig. 4(a)**, **(b)**, and **Fig. 5** shows the Step 1, Step 2, and Step 3 of Algorithm IndependentForest, respectively. Based on the independent forest in **Fig. 5**, the $2 \times 4 - 2 = 6$ node-ISTs $T_1, T_2, \dots, T_6$ rooted at node $[0, 8]$ are constructed. See **Fig. 6**. The newly appended nodes are connected by dotted lines. In every tree, there are 10 newly appended nodes. For example, in T_1 ,

By Algorithm NodeIST, for node 0 that has even number of 1's, we have

- $[0, 2]$ and $[0, 4]$ are connected to node $[0, 1]$.
- For node 5, 6, 10, 12, 15 that have even number of 1's, we have
- $[5, 7]$ is connected to node $[5, 1]$.
- $[6, 4]$ and $[6, 2]$ are connected to node $[6, 7]$.
- $[10, 8]$ and $[10, 14]$ are connected to node $[11, 10]$.
- $[12, 8]$ and $[12, 14]$ are connected to node $[12, 13]$.
- $[15, 13]$ is connected to $[15, 11]$.

Similarly, the newly appended edges in $T_2, T_3, \dots, T_6$ are constructed.

There are 6 internally node-disjoint paths between the root node $[0, 8]$ and any other node in $L(Q_4)$. Taking node $[7, 15]$ for example, the six internally node-disjoint paths are as follows.

$T_1': [0, 8] \rightarrow [0, 1] \rightarrow [3, 1] \rightarrow [3, 7] \rightarrow [7, 15]$.

$T_2': [0, 8] \rightarrow [0, 2] \rightarrow [6, 2] \rightarrow [6, 7] \rightarrow [7, 15]$.

$T_3': [0, 8] \rightarrow [0, 4] \rightarrow [5, 4] \rightarrow [5, 7] \rightarrow [7, 15]$.

$T_4': [0, 8] \rightarrow [8, 9] \rightarrow [11, 9] \rightarrow [11, 15] \rightarrow [7, 15]$.

$T_5': [0, 8] \rightarrow [8, 10] \rightarrow [14, 10] \rightarrow [14, 15] \rightarrow [7, 15]$.

$T_6': [0, 8] \rightarrow [8, 12] \rightarrow [13, 12] \rightarrow [13, 15] \rightarrow [7, 15]$.

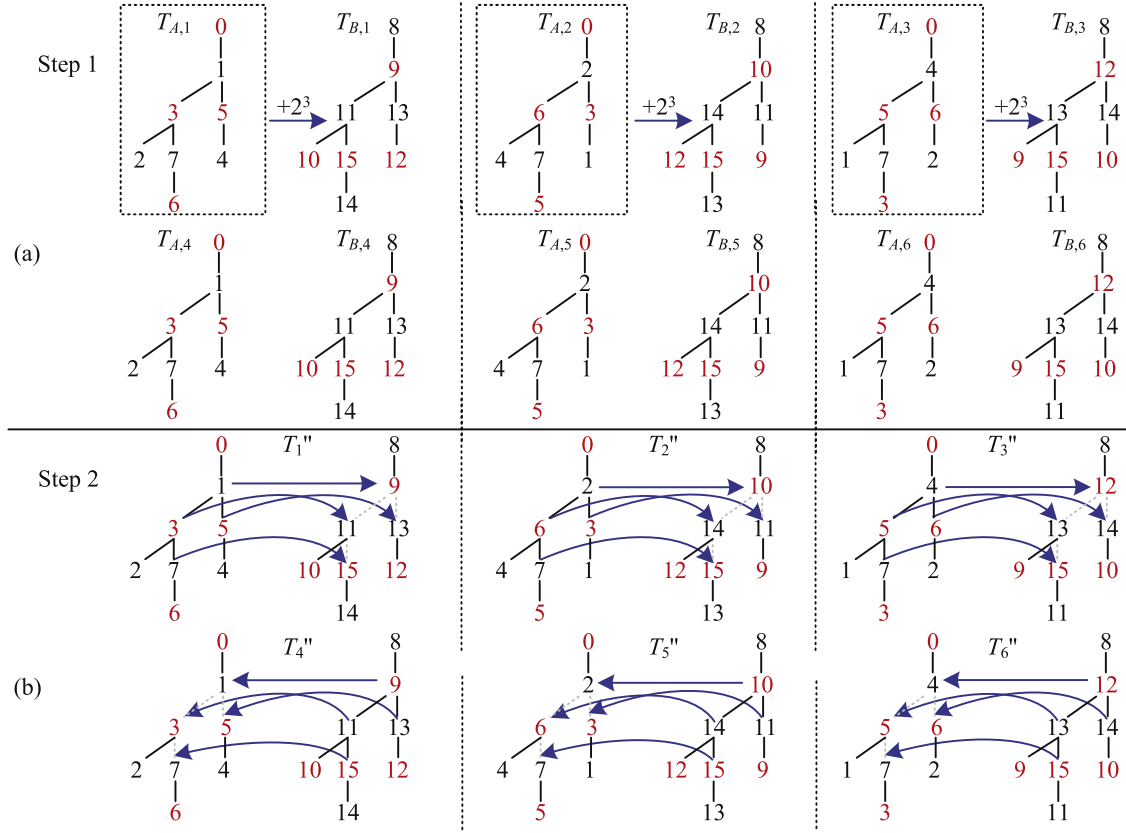


Fig. 4. Illustration of Algorithm IndependentForest(a) Step 1; (b) Step 2.

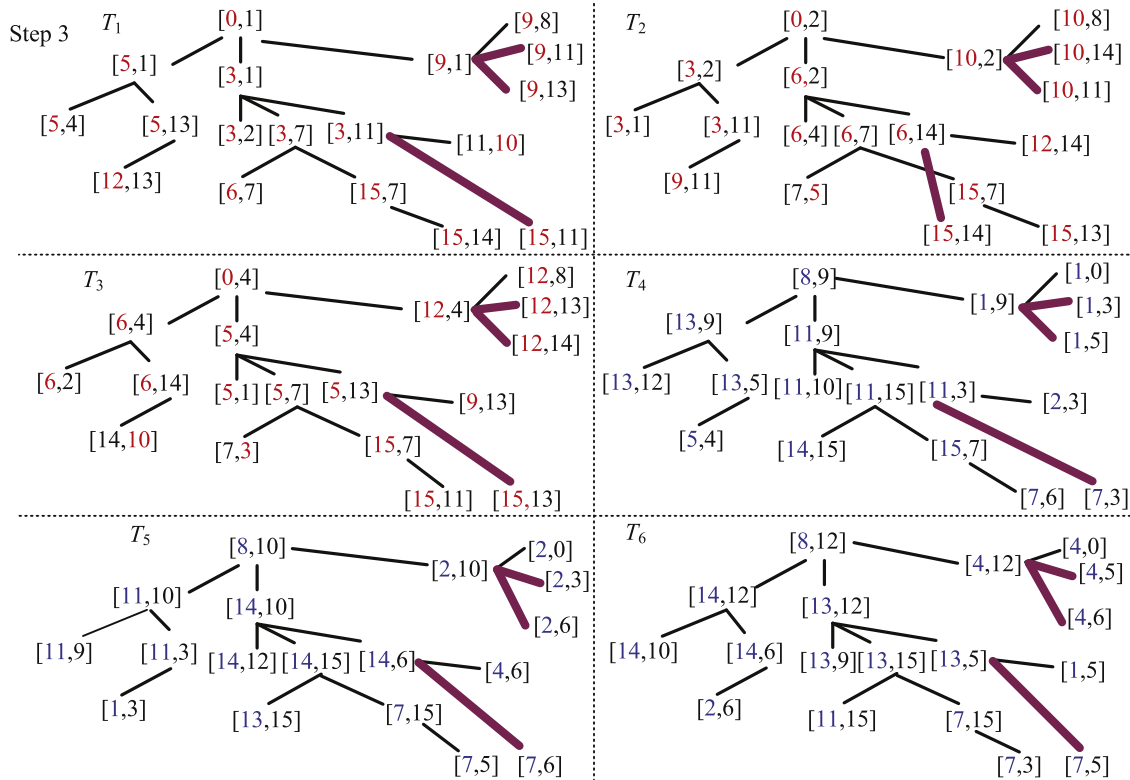


Fig. 5. Illustration of Algorithm IndependentForest Step 3.

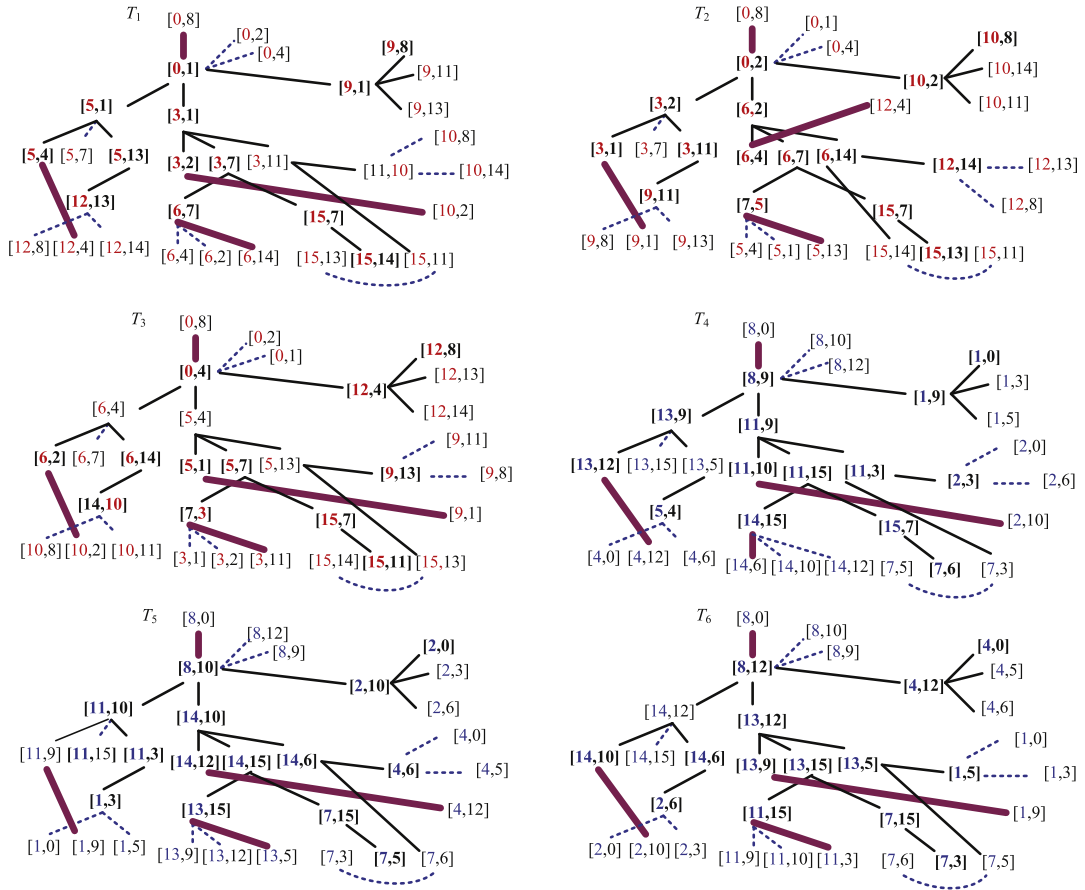


Fig. 6. Six trees $T_1, T_2, \dots, T_6$ are node-ISTs rooted at node $[0,8]$.

3.3. Correctness of node-ISTs obtained by algorithm NodeIST

By Algorithm NodeIST, every edge of G is contained in T_i for $i = 1, 2, \dots, n$. Thus, we have the following lemma.

Lemma 3.5. T_i obtained by Algorithm NodeIST is a spanning tree on $L(Q_n)$ rooted at $[0, 2^{n-1}]$ for any integer i with $1 \leq i \leq 2n - 2$.

Proof. By Step 1 of Algorithm NodeIST, $T_{B,i}$ contains all the nodes in $V(Q_{n-1}^1)$. All the edge-nodes in $T_{A,i}$ and $T_{B,i}$ plus all the edge-nodes between the internal nodes in $T_{A,i}$ and $T_{B,i}$ are formed in T'_i , for $i=1, 2, \dots, 2n-2$. By Theorem 3.1, $T'_1, T'_2, \dots, T'_{2n-2}$ form an independent forest on $L(Q_n)$ containing $2n-2$ trees. Clearly, T'_i is a tree and a subgraph of $L(Q_n)$.

By Step 2 of Algorithm NodeIST, we can easily verify that the following two results hold.

(1) The root node $[0, 2^{n-1}]$ is connected to node $[0, 2^0], [0, 2^1], \dots, [0, 2^{n-2}], [2^{n-1}, 2^0 + 2^{n-1}], [2^{n-1}, 2^1 + 2^{n-1}], \dots, [2^{n-1}, 2^{n-2} + 2^{n-1}]$ in $T_1^*, T_2^*, \dots, T_{2n-2}^*$, respectively.

(2) The edges $([0, 2^{n-1}], [0, 2^0]), ([0, 2^{n-1}], [0, 2^1]), \dots, ([0, 2^{n-1}], [0, 2^{n-2}]), ([0, 2^{n-1}], [2^{n-1}, 2^0 + 2^{n-1}]), ([0, 2^{n-1}], [2^{n-1}, 2^1 + 2^{n-1}]), \dots, ([0, 2^{n-1}], [2^{n-1}, 2^{n-2} + 2^{n-1}])$ are used uniquely in $T_1^*, T_2^*, \dots, T_{2n-2}^*$, respectively.

(3) All the edges joining Q_{n-1}^0 and Q_{n-1}^1 have been transformed into the edge-nodes in T_i^* for $i=1, 2, \dots, 2n-2$.

Furthermore, by Step 3 of Algorithm NodeIST, the rest of the nodes in $V(L(Q_n)) \setminus T_i^*$ are adjacent to the nodes with the same member in the set containing all the nodes with even number of 1's (resp., odd number of 1's) in T_i^* if $1 \leq i \leq n-1$ (resp., $n \leq i \leq 2n-2$). The obtained tree is T_i . As a result, T_i is a spanning tree on $L(Q_n)$. $\square$

Suppose that T is a tree rooted at node $[u, v]$ and $[x, y]$ is an arbitrary node in the set $V(G) \setminus \{[u, v]\}$. We use $\text{path}([u, v], [x, y], T)$ to denote the node set of the path started from $[u, v]$ and ended at $[x, y]$ in T . By the definition of independent spanning trees, we present the following lemma.

Lemma 3.6. Let T_A and T_B be two different spanning trees rooted at node a on G . If for every node b in G , $b \neq a$, $V(\text{path}(a, b, T_A)) \cap V(\text{path}(a, b, T_B)) = \{a, b\}$ and $E(\text{path}(a, b, T_A)) \cup E(\text{path}(a, b, T_B)) \supset \{a, b\}$, then T_A and T_B are node-independent.

Now we prove that the $2n-2$ trees obtained by Algorithm NodeIST are $2n-2$ node-independent spanning trees.

Theorem 3.2. $T_1, T_2, \dots, T_{2n-2}$ obtained by Algorithm NodeIST are $2n-2$ node-ISTs rooted at node $[0, 2^{n-1}]$ on $L(Q_n)$.

Proof. By Lemma 3.5, T_i obtained by Algorithm NodeIST is a spanning tree rooted at $[0, 2^{n-1}]$ on $L(Q_n)$ for any integer i with $1 \leq i \leq 2n-2$.

Since $T'_1, T'_2, \dots, T'_{2n-2}$ form an independent forest on $L(Q_n)$ containing $2n-2$ trees by Theorem 3.1, we can verify that

$T_1[V(T'_1) \cup \{[0, 2^{n-1}]\}], T_2[V(T'_2) \cup \{[0, 2^{n-1}]\}], \dots, T_{2n-2}[V(T'_{2n-2}) \cup \{[0, 2^{n-1}]\}]$ form an independent forest rooted at $[0, 2^{n-1}]$ on $L(Q_n)$ containing $2n-2$ trees, which are proper subgraphs of $T_1, T_2, \dots, T_{2n-2}$, respectively.

Without loss of generality, suppose that $1 \leq i < j \leq 2n-2$. Now, we only need to prove that the nodes created by Step 2 and Step 3 of Algorithm NodeIST satisfy the condition of the algorithm. Then, we have the following cases.

Case 1. The new nodes are created by Step 2 of Algorithm NodeIST. For any two integers i, j with $1 \leq i < j \leq 2n-2$ and the new node $[u, v]$ with u, v , we have the following subcases.

Algorithm 2 NodeIST.

Input: $n - 1$ node-ISTs $T_{A,1}, T_{A,2}, \dots, T_{A,n-1}$ rooted at 0 on Q_{n-1}^0 .

Output: $2n - 2$ node-ISTs $T_1, T_2, \dots, T_{2n-2}$ rooted at node $[0, 2^{n-1}]$ on $L(Q_n)$.

Begin

Step 1: Call Algorithm IndependentForest to obtain a forest on $L(Q_n)$ containing $2n - 2$ trees $T_1', T_2', \dots, T_{2n-2}'$.

Step 2: Construct tree T_k^* as follows.

- 1: **for** $k = 1$ to $n - 1$ **do**
- 2: **in parallel**
- 3: For any leaf node x in $T_{A,k}$, connect node $[x, \text{parent}(x, T_{A,k})]$ with node $[x, x + 2^{n-1}]$ in T_k' .
- 4: **end for**
- 5: **for** $k = n$ to $2n - 2$ **do**
- 6: **in parallel**
- 7: For any leaf node y in $T_{B,k}$, connect node $[y, \text{parent}(y, T_{A,k})]$ with node $[y, y - 2^{n-1}]$ in T_k' .
- 8: **end for**
- 9: **Step 3:** Construct tree T_k as follows.
- 10: **for** $k = 1$ to $n - 1$ **do**
- 11: **in parallel**
- 12: For any rest node $[u, v]$ in $L(Q_n)$, suppose that u has even number of 1's.
- 13: **if** $u = 0$ **then**
- 14: Connect $[u, 2^{k-1}]$ with $[u, v]$.
- 15: **else**
- 16: **if** $u_{n-1} = 0$ **then**
- 17: Connect $[u, \text{parent}(u, T_{A,k})]$ with $[u, v]$.
- 18: **else**
- 19: Connect $[u, \text{parent}(u, T_{B,k})]$ with $[u, v]$.
- 20: **end if**
- 21: **end if**
- 22: **for** $k = n$ to $2n - 2$ **do**
- 23: **in parallel**
- 24: For any rest node $[u, v]$ in $L(Q_n)$, suppose that u has odd number of 1's.
- 25: **if** $u = 2^{n-1}$ **then**
- 26: connect $[u, 2^{n-1} + 2^{k-n}]$ with $[u, v]$.
- 27: **else**
- 28: **if** $u_{n-1} = 1$ **then**
- 29: connect $[u, \text{parent}(u, T_{B,k})]$ with $[u, v]$.
- 30: **else**
- 31: connect $[u, \text{parent}(u, T_{A,k})]$ with $[u, v]$.
- 32: **end if**
- 33: **end if**
- 34: **end for**
- 35: **End**

Case 1.1. $1 \leq i < j \leq n - 1$. By Algorithm NodeIST, $u \in V(T_{A,i})$ and the path from 0 to u in $T_{A,j}$ is denoted as follows.

$P': 0, a_1, a_2, a_3, \dots, a_{k-1}, a_k = u$.

The path from 0 to u in $T_{A,j}$ is denoted as:

$Q': 0, b_1, b_2, b_3, \dots, b_{l-1}, b_l = u$.

Since $T_{A,i}$ and $T_{A,j}$ are node-independent, P' and Q' are internally node-disjoint.

We further construct paths P and Q in T_i and T_j as follows.

$P: [0, 2^{n-1}], [0, a_1], [a_1, a_2], [a_2, a_3], \dots, [a_{k-1}, a_k = u], [u, v]$.

$Q: [0, 2^{n-1}], [0, b_1], [b_1, b_2], [b_2, b_3], \dots, [b_{l-1}, b_l = u], [u, v]$.

Based on the above discussion, we can verify that paths P and Q are two internally node-disjoint paths.

Case 1.2. $1 \leq i \leq n - 1$ and $n \leq j \leq 2n - 2$. By Algorithm NodeIST, the path from 0 to u in $T_{A,i}$ is denoted as follows.

$P': 0, a_1, a_2, a_3, \dots, a_{k-1}, a_k = u$.

The path from 2^{n-1} to u in $T_{B,j}$ is denoted as:

$Q': 2^{n-1}, b_1, b_2, b_3, \dots, b_{l-1}, b_l = v$.

Here, $b_1, b_2, b_3, \dots, b_{l-1}, b_l > 2^{n-1}$ and $a_1, a_2, a_3, \dots, a_{k-1}, a_k < 2^{n-1}$. Clearly, P' and Q' are internally node-disjoint.

We further construct paths P and Q in T_i and T_j as follows.

$P: [0, 2^{n-1}], [0, a_1], [a_1, a_2], [a_2, a_3], \dots, [a_{k-1}, a_k = u], [u, v]$.

$Q: [0, 2^{n-1}], [2^{n-1}, b_1], [b_1, b_2], [b_2, b_3], \dots, [b_{l-1}, b_l = v], [u, v]$.

Based on the above discussion, we can verify that P and Q are two internally node-disjoint paths.

Case 1.3. $n \leq i < j \leq 2n - 2$. Since the construction of $T_n, T_{n+1}, \dots, T_{2n-2}$ (Lines 22–34 in Algorithm NodeIST) is similar to $T_1, T_2, \dots, T_{n-1}$ (Lines 9–21 in Algorithm NodeIST), respectively, the proof is similar to that in **Case 1.1**.

Case 2. The new nodes are created by **Step 3** of Algorithm NodeIST. By **Definition 2.1**, we can assume that u is a node with even number of 1's and v is a node with odd number of 1's. We have the following subcases.

Case 2.1. $1 \leq i < j \leq n - 1$. Without loss of generality, suppose that $[u, v]$ is a new node in T_i . We have the following subcases.

Case 2.1.1. $u = 0$. $[u, v]$ is adjacent to node $[0, 2^{i-1}]$ in T_i .

If $v = 2^{j-1}$, the path P from $[0, 2^{n-1}]$ to $[u, v]$ in T_i is $[0, 2^{n-1}], [0, 2^{i-1}], [u, v = 2^{j-1}]$, the path Q from $[0, 2^{n-1}]$ to $[u, v]$ in T_j is $[0, 2^{n-1}], [u, v = 2^{j-1}]$. Otherwise, the path P from $[0, 2^{n-1}]$ to $[u, v]$ in T_i is $[0, 2^{n-1}], [0, 2^{i-1}], [u, v]$, the path Q from $[0, 2^{n-1}]$ to $[u, v]$ in T_j is $[0, 2^{n-1}], [u, 2^{j-1}], [u, v]$.

Clearly, P and Q are also internally node-disjoint.

Case 2.1.2. $u \neq 0$ and $u_{n-1} = 0$.

The path from 0 to u in $T_{A,i}$ is denoted as follows.

$P': 0, a_1, a_2, a_3, \dots, a_{k-1}, a_k = u$.

The path from 0 to u in $T_{B,i}$ is denoted as:

$Q': 0, b_1, b_2, b_3, \dots, b_{l-1}, b_l = u$.

Clearly, P' and Q' are internally node-disjoint.

We further construct paths P and Q in T_i and T_j as follows.

$P: [0, 2^{n-1}], [0, a_1], [a_1, a_2], [a_2, a_3], \dots, [a_{k-1}, a_k = u], [u, v]$.

If $b_{l-1} = v$, we construct Q as follows.

$Q: [0, 2^{n-1}], [0, b_1], [b_1, b_2], [b_2, b_3], \dots, [b_{l-1}, b_l = u]$.

Otherwise,

$Q: [0, 2^{n-1}], [0, b_1], [b_1, b_2], [b_2, b_3], \dots, [b_{l-1}, b_l = u], [u, v]$.

Clearly, P and Q are two internally node-disjoint paths.

Case 2.1.3. $u \neq 0$ and $u_{n-1} \neq 0$.

The path from 0 to u in $T_{A,i}$ is denoted as follows.

$P': 0, a_1, a_2, a_3, \dots, a_{k-1}, a_{k-1} + 2^{n-1}, a_k + 2^{n-1} = u$.

The path from 0 to u in $T_{B,i}$ is denoted as:

$Q': 0, b_1, b_2, b_3, \dots, b_{l-1}, b_{l-1} + 2^{n-1}, b_l + 2^{n-1} = u$.

Clearly, P' and Q' are internally node-disjoint.

We further construct path P as follows.

$P: [0, 2^{n-1}], [0, a_1], [a_1, a_2], [a_2, a_3], \dots, [a_{k-1}, a_{k-1} + 2^{n-1}, a_k + 2^{n-1} = u], [u, v]$.

The path from 0 to u in $T_{B,i}$ is denoted as:

If $b_{l-1} + 2^{n-1} = v$, we construct Q as follows.

$Q: [0, 2^{n-1}], [0, b_1], [b_1, b_2], [b_2, b_3], \dots, [b_{l-1}, b_{l-1} + 2^{n-1} = v], [b_l + 2^{n-1} = u, v]$.

Otherwise,

$Q: [0, 2^{n-1}], [0, b_1], [b_1, b_2], [b_2, b_3], \dots, [b_{l-1}, b_{l-1} + 2^{n-1}], [b_{l-1} + 2^{n-1}, b_l + 2^{n-1} = u], [u, v]$.

Clearly, P and Q are two internally node-disjoint paths.

Case 2.2. $1 \leq i \leq n - 1$ and $n \leq j \leq 2n - 2$. We have the following subcases.

Case 2.2.1. $[u, v]$ is a new node in T_i . Without loss of generality, suppose that u is the node with even number of 1's and v is the node with odd number of 1's.

Case 2.2.1.1. $u = 0$. $[u, v]$ is adjacent to node $[0, 2^{i-1}]$ in T_i .

If $v = 2^{j-n}$, the path P from $[0, 2^{n-1}]$ to $[u, v]$ in T_i is $[0, 2^{n-1}], [0, 2^{i-1}], [u = 0, v = 2^{j-n}]$, the path Q from $[0, 2^{n-1}]$ to $[u, v]$ in T_j is $[2^{n-1}, 2^{n-1} + 2^{j-n}], [2^{n-1} + 2^{j-n}, 2^{j-n} = v], [u, v = 2^{j-n}]$. Clearly,

P and Q are internally node-disjoint. Otherwise, the path P from $[0, 2^{n-1}]$ to $[u, v]$ in T_i is $[0, 2^{n-1}]$, $[0, 2^{i-1}]$, $[u, v]$, the path Q from $[0, 2^{n-1}]$ to $[u, v]$ in T_j is $[2^{n-1}, 2^{n-1} + 2^{j-n}]$, $[2^{n-1} + 2^{j-n}, 2^{j-1}]$, $[2^{j-1} = v, u = 0]$. P and Q are also internally node-disjoint.

Case 2.2.1.2. $u \neq 0$ and $u_{n-1} = 0$.

The path from 0 to u in $T_{A,i}$ is denoted as follows.

P' : $0, a_1, a_2, a_3, \dots, a_{k-1}, a_k = u$.

The path from 0 to v in $T_{B,i}$ is denoted as follow:

Q' : $2^{n-1}, b_1, b_2, b_3, \dots, b_{l-1} - 2^{n-1}, b_l - 2^{n-1} = v$.

We further construct paths P and Q in T_i and T_j as follows.

P : $[0, 2^{n-1}]$, $[0, a_1]$, $[a_1, a_2]$, $[a_2, a_3]$, $\dots$, $[a_{k-1}, a_k = u]$, $[u, v]$.

If $b_{l-1} - 2^{n-1} = u$, we construct Q as follows.

Q : $[0, 2^{n-1}]$, $[2^{n-1}, b_1]$, $[b_1, b_2]$, $[b_2, b_3]$, $\dots$, $[b_{l-2}, b_{l-1}]$, $[b_{l-1} - 2^{n-1} = u, b_l - 2^{n-1} = v]$.

Otherwise,

Q : $[0, 2^{n-1}]$, $[2^{n-1}, b_1]$, $[b_1, b_2]$, $[b_2, b_3]$, $\dots$, $[b_{l-2}, b_{l-1}]$, $[b_{l-1} - 2^{n-1}, b_l - 2^{n-1} = v]$, $[u, v]$.

Since $a_1, a_2, \dots, a_k < 2^{n-1}$ and $b_1, b_2, \dots, b_l > 2^{n-1}$, which implies that paths P and Q are internally node-disjoint.

Case 2.2.1.3. $u \neq 0$ and $u_{n-1} \neq 0$.

The path from 0 to u in $T_{A,i}$ is denoted as follows.

P' : $0, a_1, a_2, a_3, \dots, a_{k-1}, a_{k-1} + 2^{n-1}, a_k + 2^{n-1} = u$.

The path from 2^{n-1} to v in $T_{B,i}$ is denoted as:

Q' : $2^{n-1}, b_1, b_2, b_3, \dots, b_{l-1}, b_l = v$.

We further construct path P as follows.

P : $[0, 2^{n-1}]$, $[0, a_1]$, $[a_1, a_2]$, $[a_2, a_3]$, $\dots$, $[a_{k-1}, a_{k-1} + 2^{n-1}]$, $[a_{k-1} + 2^{n-1}, a_k + 2^{n-1} = u]$, $[u, v]$.

The path from 0 to v in $T_{B,i}$ is denoted as:

If $b_{l-1} = u$, we construct Q as follows.

Q : $[0, 2^{n-1}]$, $[2^{n-1}, b_1]$, $[b_1, b_2]$, $[b_2, b_3]$, $\dots$, $[b_{l-1}, b_{l-1} = u]$, $[b_{l-1} = u, b_l = v]$.

Otherwise,

Q : $[0, 2^{n-1}]$, $[2^{n-1}, b_1]$, $[b_1, b_2]$, $[b_2, b_3]$, $\dots$, $[b_{l-1}, b_l = v]$, $[u, v]$.

Since $a_1, a_2, \dots, a_k < 2^{n-1}$ and $b_1, b_2, \dots, b_l > 2^{n-1}$, which implies that paths P and Q are internally node-disjoint.

Case 2.2.2. $[u, v]$ is not a new node in T_i . Then, $[u, v]$ is a new node in T_j . Without loss of generality, suppose that u is the node with even number of 1's. We have the following subcases.

Case 2.2.2.1. $u = 0$. $[u, v]$ is adjacent to node $[0, 2^{i-1}]$ in T_i .

If $v = 2^{j-1}$, the path P from $[0, 2^{n-1}]$ to $[u, v]$ in T_i is $[0, 2^{n-1}]$, $[0, 2^{i-1}]$, $[u = 0, v = 2^{i-1}]$, the path Q from $[0, 2^{n-1}]$ to $[u, v]$ in T_j is $[2^{n-1}, 2^{n-1} + 2^{j-1}]$, $[2^{n-1} + 2^{j-1}, 2^{j-1} = v]$, $[u, v = 2^{j-1}]$. Clearly, P and Q are internally node-disjoint. Otherwise, the path P from $[0, 2^{n-1}]$ to $[u, v]$ in T_i is $[0, 2^{n-1}]$, $[0, 2^{i-1}]$, $[u, v]$, the path Q from $[0, 2^{n-1}]$ to $[u, v]$ in T_j is $[2^{n-1}, 2^{n-1} + 2^{j-1}]$, $[2^{n-1} + 2^{j-1}, 2^{j-1}]$, $[2^{j-1} = v, u = 0]$. P and Q are also internally node-disjoint.

Case 2.2.2.2. $u \neq 0$ and $u_{n-1} = 0$. The path from 0 to u in $T_{A,i}$ is denoted as follows.

P' : $0, a_1, a_2, a_3, \dots, a_{k-1}, a_k = u$.

The path from 0 to v in $T_{B,i}$ is denoted as:

Q' : $2^{n-1}, b_1, b_2, b_3, \dots, b_{l-1} - 2^{n-1}, b_l - 2^{n-1} = v$.

If $a_{k-1} = v$, we construct path P as

P : $[0, 2^{n-1}]$, $[0, a_1]$, $[a_1, a_2]$, $[a_2, a_3]$, $\dots$, $[a_{k-1} = v, a_k = u]$.

Otherwise,

P : $[0, 2^{n-1}]$, $[0, a_1]$, $[a_1, a_2]$, $[a_2, a_3]$, $\dots$, $[a_{k-1} = v, a_k = u]$, $[u, v]$.

we construct Q as

Q : $[0, 2^{n-1}]$, $[2^{n-1}, b_1]$, $[b_1, b_2]$, $[b_2, b_3]$, $\dots$, $[b_{l-2}, b_{l-1} - 2^{n-1}]$, $[b_{l-1} - 2^{n-1}, b_l - 2^{n-1} = v]$, $[u, v]$.

Since $a_1, a_2, \dots, a_k < 2^{n-1}$ and $b_1, b_2, \dots, b_l > 2^{n-1}$, which implies that paths P and Q are internally node-disjoint.

Case 2.2.2.3. $u \neq 0$ and $u_{n-1} \neq 0$.

The path from 0 to u in $T_{A,i}$ is denoted as follows.

P' : $0, a_1, a_2, a_3, \dots, a_{k-1}, a_{k-1} + 2^{n-1}, a_k + 2^{n-1} = u$.

The path from 2^{n-1} to v in $T_{B,i}$ is denoted as:

Q' : $2^{n-1}, b_1, b_2, b_3, \dots, b_{l-1}, b_l = v$.

We further construct path P as follows. If $a_{k-1} + 2^{n-1} = v$, then

P : $[0, 2^{n-1}]$, $[0, a_1]$, $[a_1, a_2]$, $[a_2, a_3]$, $\dots$, $[a_{k-1}, a_{k-1} + 2^{n-1}]$, $[a_{k-1} + 2^{n-1} = v, a_k + 2^{n-1} = u]$.

Otherwise,

P : $[0, 2^{n-1}]$, $[0, a_1]$, $[a_1, a_2]$, $[a_2, a_3]$, $\dots$, $[a_{k-1}, a_{k-1} + 2^{n-1}]$, $[a_{k-1} + 2^{n-1} = v, a_k + 2^{n-1} = u]$, $[u, v]$.

The path from 0 to v in $T_{B,i}$ is denoted as:

Q : $[0, 2^{n-1}]$, $[2^{n-1}, b_1]$, $[b_1, b_2]$, $[b_2, b_3]$, $\dots$, $[b_{l-1}, b_{l-1} = u]$, $[b_{l-1}, b_l = v]$, $[u, v]$.

Since $a_1, a_2, \dots, a_k < 2^{n-1}$ and $b_1, b_2, \dots, b_l > 2^{n-1}$, which implies that paths P and Q are internally node-disjoint.

Case 2.3. $n \leq i < j \leq 2n - 2$. The proof is similar to that in **Case 2.1**.

As a result, the theorem holds. $\square$

Based on the above discussion, we can verify that the following theorem holds.

Theorem 3.3. The height of T_i is $n + 2$ if n is odd, otherwise, $n + 1$, for $i = 1, 2, \dots, 2n - 2$.

For the structure homogeneity of node-ISTs, we have the following theorem by Algorithm NodeIST.

Theorem 3.4. $T_1, T_2, \dots, T_{2n-2}$ are isomorphic to each other.

Since the node-ISTs $T_1, T_2, \dots, T_{2n-2}$ are constructed in parallel mode, we only need to consider the construction of T_1 . By Algorithm NodeIST, the time complexity is equal to the number of nodes in T_1 (the number of nodes in $L(Q_n)$), which are transformed from the edges in Q_n . Then, we have the following theorem.

Theorem 3.5. The time complexity of Algorithm NodeIST is $O(N)$, where $N = n \times 2^{n-1}$ is the number of nodes in $L(Q_n)$.

3.4. Discussion of node/edge-connectivity and node/edge-wide-diameter in $L(Q_n)$ based on node-ISTs

If the node/edge-connectivity of a graph G is k , it can tolerate up to $k - 1$ faulty nodes/edges. Any two nodes can always communicate with each other reliably. It is well known that $\kappa(Q_n) = \lambda(Q_n) = n$. That is, Q_n has the maximum node/edge-connectivity. We will prove that $L(Q_n)$ also has the maximum node/edge-connectivity.

Theorem 3.6. $\kappa(L(Q_n)) = \lambda(L(Q_n)) = 2n - 2$.

Proof. Since $L(Q_n)$ is $(2n-2)$ -regular, we have $\kappa(L(Q_n)), \lambda(L(Q_n)) \leq 2n - 2$. For a graph G , it is well-known that $\kappa(G) \leq \lambda(G) \leq \delta(G)$ [13] and $\lambda(Q_n) = n$. Thus, it is sufficient to prove that $\kappa(L(Q_n)) \geq 2n - 2$.

Since there exist $2n - 2$ node-ISTs rooted at any node on $L(Q_n)$, there always exist $2n - 2$ internally node-disjoint paths between any two nodes. This implies that it is also a connected graph, if we remove arbitrary $2n - 3$ faulty edges from $L(Q_n)$. Then, $\kappa(L(Q_n)) = 2n - 2$.

In summary, $\kappa(L(Q_n)) = \lambda(L(Q_n)) = 2n - 2$. $\square$

The k -edge-wide-distance between x and y is the minimum integer l such that there exist k edge-disjoint paths between x and y whose lengths are at most l . For G , the k -edge-wide-diameter $d_k^e(G)$ is the maximum value of the k -edge-wide-distance between two nodes in G [28].

Based on Theorems 3.2 and 3.4, we have the upper bounds as the following theorem.

Theorem 3.7. For the $(2n - 2)$ -edge-wide-diameter $d_{2n-2}^e(L(Q_n))$, we have

$$d_{2n-2}^e(L(Q_n)) \leq \begin{cases} n + 2, & \text{if } n \text{ is odd,} \\ n + 1, & \text{otherwise.} \end{cases}$$

Since $\kappa(L(Q_n)) = 2n - 2$, by the Menger Theorem, we can find $2n - 2$ internally disjoint paths between given nodes u and v such that the length of the longest path among the $2n - 2$ disjoint paths is minimum, denoted by $d_{2n-2}(u, v)$, among all possible choice of $2n - 2$ internally disjoint paths. The wide diameter is defined as the maximum of $d_{2n-2}(u, v)$ over all $u, v \in V(L(Q_n))$, denoted as $D_{2n-2}(L(Q_n))$. Based on Theorem 3.4, we can have the following theorem.

Theorem 3.8. For the wide diameter $D_{2n-2}(L(Q_n))$, we have

$$D_{2n-2}(L(Q_n)) \leq \begin{cases} n + 2, & \text{if } n \text{ is odd,} \\ n + 1, & \text{otherwise.} \end{cases}$$

4. Conclusions

In this paper, we discussed the construction of 2^{n-1} complete graphs in $L(Q_n)$ by drawing the nodes in the n -dimensional Q_n in black and white. Then, based on the complete graphs and $n - 1$ node-ISTs rooted at 0 on Q_{n-1}^0 , we presented an algorithm to construct an independent forest containing $2n - 2$ trees on $L(Q_n)$. Next, we presented the construction method of $2n - 2$ isomorphic node-ISTs rooted at node $[0, 2^{n-1}]$ on $L(Q_n)$, based on the independent forest. The corresponding algorithm with time complexity $O(N)$ is proposed and its correctness proved, where $N = n \times 2^{n-1}$ is the number of nodes in $L(Q_n)$. We have pointed out that the construction of $2n - 2$ node-ISTs on $L(Q_n)$ is a method to prove the node/edge-connectivity and the upper bound of $(2n - 2)$ -edge-wide-diameter/wide diameter of $L(Q_n)$. We conclude the paper by posing the following open question:

Given an n -node-connected network G with $n \geq 1$, if there exist n node-ISTs rooted at an arbitrary node on G , does it imply that there exist an independent spanning forest containing n spanning trees rooted at n nodes differ from one another?

Declaration of competing interest

No author associated with this paper has disclosed any potential or pertinent conflicts which may be perceived to have impending conflict with this work. For full disclosure statements refer to <https://doi.org/10.1016/j.jpdc.2019.08.006>.

Acknowledgments

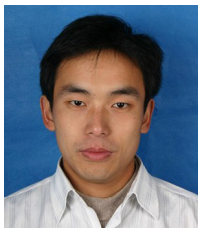
This work is supported by National Natural Science Foundation of China (Nos. 61572337, 61872257, and 61702351), China Postdoctoral Science Foundation Funded Project (No. 2015M581858), the Natural Science Foundation of the Jiangsu Higher Education Institutions of China (Nos. 18KJA520009 and 17KJB520036), Opening Foundation of Jiangsu High Technology Research Key Laboratory for Wireless Sensor Networks (No. WSNLBKF201701), the Jiangsu Planned Projects for Postdoctoral Research Funds (No. 1501089B), and Postgraduate Research & Practice Innovation Program of Jiangsu Province (No. KYCX17_2005).

We would like to express our sincere thanks to Professor Dajin Wang for his constructive suggestions, as well as the anonymous referees comments that greatly helped improve the original manuscript.

References

- [1] F. Bao, Y. Funyu, Y. Hamada, Y. Igarashi, Reliable broadcasting and secure distributing in channel networks, IEICE Trans. Fundam. Electron. Commun. Comput. Sci. E81-A (1998) 796–806.
- [2] F. Bao, Y. Igarashi, S.R. Öhring, Reliable broadcasting in product networks, Discrete Appl. Math. 83 (1–3) (1998) 3–20.
- [3] F. Bonomom, G. Durán, M.D. Safe, A.K. Wagler, Clique-perfectness of complements of line graphs, Discrete Appl. Math. 186 (1) (2015) 19–44.
- [4] J.-M. Chang, T.-J. Yang, J.-S. Yang, A parallel algorithm for constructing independent spanning trees in twisted cubes, Discrete Appl. Math. 219 (2017) 74–82.
- [5] Y.-S. Chen, C.-Y. Chiang, C.-Y. Chen, Multi-node broadcasting in all-ported 3-d wormhole-routed torus using an aggregation-then-distribution strategy, J. Syst. Archit. 50 (9) (2004) 575–589.
- [6] B. Cheng, J. Fan, X. Jia, Dimensional-permutation-based independent spanning trees in bijective connection networks, IEEE Trans. Parallel Distrib. Syst. 26 (1) (2015) 45–53.
- [7] B. Cheng, J. Fan, X. Jia, J. Wang, Dimension-adjacent trees and parallel construction of independent spanning trees on crossed cubes, J. Parallel Distrib. Comput. 73 (5) (2013) 641–652.
- [8] B. Cheng, J. Fan, X. Jia, S. Zhang, B. Chen, Constructive algorithm of independent spanning trees on Möbius cubes, Comput. J. 56 (11) (2013) 1347–1362.
- [9] B. Cheng, J. Fan, Q. Lyu, J. Zhou, Z. Liu, Constructing independent spanning trees with height n on the n -dimensional crossed cube, Future Gener. Comput. Syst. 87 (2018) 404–415.
- [10] B. Cheng, J. Fan, J. Yang, X. Wang, Circular dimensional-permutations and reliable broadcasting for hypercubes and Möbius cubes, in: NPC, 2013, pp. 232–244.
- [11] J. Cheriyan, S.N. Maheshwari, Finding nonseparating induced cycles and independent spanning trees in 3-connected graphs, J. Algorithms 9 (4) (1988) 507–537.
- [12] S. Curran, O. Lee, X. Yu, Finding four independent trees, SIAM J. Comput. 35 (5) (2006) 1023–1058.
- [13] R. Diestel, Graph Theory, fourth ed., Springer-Verlag, Heidelberg, 2012.
- [14] F. Dong, W. Yan, Expression for the number of spanning trees of line graphs of arbitrary connected graphs, J. Graph Theory 85 (1) (2017) 74–93.
- [15] J. Fan, X. Jia, X. Lin, Embedding of cycles in twisted cubes with edge-pancyclic, Algorithmica 51 (3) (2008) 264–282.
- [16] A. Gopalan, S. Ramasubramanian, On constructing three edge independent spanning trees, in: Manuscript, 2011, (see <http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.114667119&rank=1>).
- [17] A. Gopalan, S. Ramasubramanian, A counterexample for the proof of implication conjecture on independent spanning trees, Inform. Process. Lett. 113 (14–16) (2013) 522–526.
- [18] J. Guo, F. Liu, J.C.S. Lui, Hai. Jin, Hai jin fair network bandwidth allocation in iaas datacenters via a cooperative game approach, IEEE/ACM Trans. Netw. 24 (2) (2016) 873–886.
- [19] J. Guo, F. Liu, T. Wang, J.C.S. Lui, Pricing intra-datacenter networks with over-committed bandwidth guarantee, in: USENIX Annual Technical Conference, 2017, pp. 69–81.
- [20] D.J. Harvey, D.R. Wood, Treewidth of the line graph of a complete graph, J. Graph Theory 79 (1) (2015) 48–54.
- [21] T. Hasunuma, Structural properties of subdivided-line graphs, J. Discrete Algorithms 31 (2015) 69–86.
- [22] A. Hoyer, R. Thomas, Four edge-independent spanning trees, SIAM J. Discrete Math. 32 (1) (2018) 233–248.
- [23] A. Huck, Independent trees in planar graphs, Graphs Combin. 15 (1) (1999) 29–77.
- [24] A. Itai, M. Rodeh, The multi-tree approach to reliability in distributed networks, Inform. and Comput. 79 (1) (1988) 43–59.
- [25] S. Khuller, B. Schieber, On independent spanning trees, Inform. Process. Lett. 42 (6) (1992) 321–323.
- [26] J.-S. Kim, H.-O. Lee, E. Cheng, L. Lipták, Independent spanning trees on even networks, Inform. Sci. 181 (13) (2011) 2892–2905.
- [27] J.-S. Kim, H.-O. Lee, E. Cheng, L. Lipták, Optimal independent spanning trees on odd graphs, J. Supercomput. 56 (2) (2011) 212–225.
- [28] T. Kojima, K. Ando, A. Kaneko, Edge-wide-diameter of graphs with diameter d , Ann. Comb. 6 (1) (2002) 57–64.
- [29] H. Li, W. He, W. Yang, Y. Bai, A note on edge-disjoint hamilton cycles in line graphs, Graphs Combin. 32 (2016) 741–744.
- [30] X.-J. Li, B. Liu, M. Ma, J.-M. Xu, Many-to-many disjoint paths in hypercubes with faulty vertices, Discrete Appl. Math. 217 (2017) 229–242.
- [31] D. Li, J. Wu, On data center network architectures for interconnecting dual-port servers, IEEE Trans. Comput. 64 (11) (2015) 3210–3222.
- [32] Y.-J. Liu, W.Y. Chou, J.K. Lan, C. Chen, Constructing independent spanning trees for locally twisted cubes, Theoret. Comput. Sci. 412 (22) (2011) 2237–2252.

- [33] F. Liu, J. Guo, X. Huang, J.C.S. Lui, eBA: Efficient bandwidth guarantee under traffic variability in datacenters, *IEEE/ACM Trans. Netw.* 25 (1) (2017) 506–519.
- [34] M. Ma, J.-M. Xu, D.B. West, The vulnerability of the diameter of the enhanced hypercubes, *Theoret. Comput. Sci.* 694 (2017) 60–65.
- [35] K. Obokata, Y. Iwasaki, F. Bao, Y. Igarashi, Independent spanning trees of product graphs and their construction, *IEICE Trans. Fundam. Electron. Commun. Comput. Sci.* E79-A (11) (1996) 1894–1903.
- [36] Y. Saad, H.G. Shultz, Topological properties of hypercube, *IEEE Trans. Comput.* 37 (7) (1988) 867–872.
- [37] G. Su, L. Xu, Topological indices of the line graph of subdivision graphs and their Schur-bounds, *Appl. Math. Comput.* 253 (2015) 395–401.
- [38] S.-M. Tang, Y.-L. Wang, Y.-H. Leu, Optimal independent spanning trees on hypercubes, *J. Inf. Sci. Eng.* 20 (1) (2004) 143–155.
- [39] T. Tian, L. Xiong, Traceability on 2-connected line graphs, *Appl. Math. Comput.* 321 (2018) 1339–1351.
- [40] Y.-C. Tseng, S.-Y. Wang, C.-W. Ho, Efficient broadcasting in wormhole-routed multicomputers: A network-partitioning approach, *IEEE Trans. Parallel Distrib. Syst.* 10 (1) (1999) 44–61.
- [41] Y. Wang, J. Fan, G. Zhou, X. Jia, Independent spanning trees on twisted cubes, *J. Parallel Distrib. Comput.* 72 (1) (2012) 58–69.
- [42] T. Wang, F. Liu, H. Xu, An efficient online algorithm for dynamic SDN controller assignment in data center networks, *IEEE/ACM Trans. Netw.* 25 (5) (2017) 2788–2801.
- [43] S.-I. Wang, F.-H. Wang, Independent spanning trees in RTCC-pyramids, *Comput. J.* 60 (1) (2017) 13–26.
- [44] T. Wang, H. Xu, F. Liu, Aemon: Information-agnostic mix-flow scheduling in data center networks, in: *Asia-Pacific Workshop on Networking*, 2017, pp. 106–112.
- [45] W. Wei, H. Gu, K. Wang, X. Yu, X. Liu, Improving cloud-based IoT services through virtual network embedding in elastic optical inter-DC networks, *IEEE Internet Things J.* 6 (1) (2019) 986–996.
- [46] D. Xiang, K. Chakrabarty, H. Fujiwara, Multicast-based testing and thermal-aware test scheduling for 3D ICs with a stacked network-on-chip, *IEEE Trans. Comput.* 65 (9) (2016) 2767–2779.
- [47] D. Xiang, Y. Zhang, Cost-effective power-aware core testing in NoCs based on a new unicast-based multicast scheme, *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 30 (1) (2011) 135–147.
- [48] J.-S. Yang, S.-M. Tang, J.-M. Chang, Y.-L. Wang, Parallel construction of optimal independent spanning trees on hypercubes, *Parallel Comput.* 33 (1) (2007) 73–79.
- [49] X. Yi, F. Liu, J. Liu, H. Jin, Building a network highway for big data: architecture and challenges, *IEEE Network* 28 (4) (2014) 5–13.
- [50] A. Zehavi, A. Itai, Three tree-paths, *J. Graph Theory* 13 (2) (1989) 175–188.



Baolei Cheng received the B.S., M.S., and Ph.D. degrees in Computer Science from the Soochow University in 2001, 2004, 2014, respectively. He is currently an Associate Professor of Computer Science with the School of Computer Science and Technology at the Soochow University, China. His research interests include parallel and distributed systems, algorithms, interconnection architectures, and software testing.



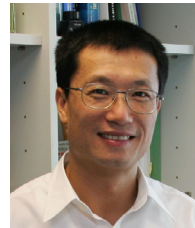
August 2012). His research interests include parallel and distributed systems, interconnection architectures, data center networks, algorithms, and graph theory.



sensor networks, mobile

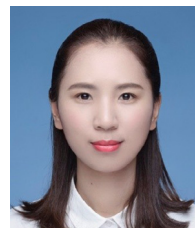
Jianxi Fan received the B.S., M.S., and Ph.D. degrees in Computer Science from the Shandong Normal University, Shandong University, and the City University of Hong Kong, China, in 1988, 1991, and 2006, respectively. He is currently a professor in the School of Computer Science and Technology at the Soochow University, China. He was a visiting scholar in the Department of Computer Science at Montclair State University (May 2017–August 2017) and a senior research fellow in the Department of Computer Science at the City University of Hong Kong (May 2012–

Cheng-Kuan Lin received his B.S. degree in science applied mathematics from the Chinese Culture University, Taipei, in 2000, and received his M.S. degree in mathematics from “National” Central University, Taipei, in 2002. He obtained his Ph.D. degree in computer science from “National” Chiao Tung University, Hsinchu, in 2011. He is currently an associate professor of computer science at School of Computer Science and Technology at the Soochow University, Suzhou. His research interests include graph theory, design and analysis of algorithms, discrete mathematics, wireless computing, and parallel and distributed computing.



etc. He is the General Chair of ACM MobiHoc 2008, TPC Co-Chair of IEEE MASS 2009, Area-Chair of IEEE INFOCOM 2010, TPC Co-Chair of IEEE GlobeCom 2010 – Ad Hoc and Sensor Networking Symp, and Panel Co-Chair of IEEE INFOCOM 2011. He is fellow of IEEE (Computer Society).

Xiaohua Jia received his BSc (1984) and MEng (1987) from University of Science and Technology of China, and DSc (1991) in Information Science from University of Tokyo. He is currently Chair Professor with Dept of Computer Science at City University of Hong Kong. His research interests include cloud computing and distributed systems, computer networks, wireless sensor networks and mobile wireless networks. Prof. Jia is an editor of *IEEE Trans. on Parallel and Distributed Systems* (2006–2009), *Wireless Networks*, *Journal of World Wide Web*, *Journal of Combinatorial Optimization*, etc. He is the General Chair of ACM MobiHoc 2008, TPC Co-Chair of IEEE MASS 2009, Area-Chair of IEEE INFOCOM 2010, TPC Co-Chair of IEEE GlobeCom 2010 – Ad Hoc and Sensor Networking Symp, and Panel Co-Chair of IEEE INFOCOM 2011. He is fellow of IEEE (Computer Society).



computing.

Xiaoyan Li received her B.S. degree in Mathematics and Statistics from the Hainan Normal University in 2011, and received her M.S. degree in Mathematics from the Fujian Normal University in 2014. She is now a Doctoral candidate at the School of Computer Science and Technology, Soochow University, China. He is currently a Visiting Postgraduate Research Student with the Dept. of Computer Science at City University of Hong Kong. Her research interests include data center networks, parallel and distributed systems, interconnection architectures, fault diagnosis, fault tolerant